

MANUAL DE INSTALACIÓN

Sistema web de venta de repuestos automotrices — Repuestos El Turco

02/07/2026

1. Objetivo del manual

Este documento explica cómo instalar y ejecutar desde cero el sistema Repuestos El Turco, incluyendo backend, frontend y base de datos poblada. El contenido fue actualizado según el README del repositorio y mantiene las capturas existentes del manual original.

2. Descripción del sistema

Repuestos El Turco es un sistema web de venta de repuestos automotrices. Incluye catálogo público, carrito de compras, checkout con Mercado Pago y panel de administración para gestionar productos, pedidos, familias, subfamilias, usuarios, compatibilidades y reportes.

- Backend: API REST desarrollada con FastAPI, SQLAlchemy y PostgreSQL.
- Frontend: sitio público y panel de administración desarrollados con React, Vite y TypeScript.
- Pagos: integración con Mercado Pago mediante Checkout Bricks.
- Base de datos: PostgreSQL con scripts SQL para estructura, carga inicial y backup completo.

3. Estructura del proyecto

```
RepuestosElTurco/
├── backend/      # API REST (FastAPI + SQLAlchemy + PostgreSQL)
├── frontend/     # Sitio público + panel admin (React + Vite)
├── database/     # Scripts SQL de la base de datos
│   ├── schema.sql      # Estructura sin datos
│   ├── seed.sql        # Datos de ejemplo/carga inicial
│   └── repuestos_el_turco_backup.sql # Backup completo (schema + datos)
└── README.md
```

4. Requisitos previos

- PostgreSQL 14 o superior.
- Python 3.11 o superior.
- Node.js 18 o superior, incluyendo npm.
- Git para clonar el repositorio.
- Cuenta de Mercado Pago Developers si se desea probar pago en línea con credenciales TEST.

5. Clonar el repositorio

```
git clone https://github.com/FernandoGF10/RepuestosElTurco.git
```

```
cd RepuestosElTurco
```

6. Instalación en Windows

Se recomienda usar PowerShell. Si el profesor instalará el sistema en Windows, esta es la sección principal que debe seguir.

6.1 Instalar PostgreSQL

- Descargar PostgreSQL desde <https://www.postgresql.org/download/windows/>.
- Durante la instalación dejar marcada la opción “Command Line Tools”, ya que incluye psql.
- Anotar la contraseña definida para el usuario postgres.
- Dejar el puerto por defecto 5432, salvo que el equipo ya use otro puerto como 5433.
- Cerrar y volver a abrir PowerShell después de instalar.

```
psql --version
```

Si PowerShell indica que psql no se reconoce como comando, agregar la carpeta bin de PostgreSQL al PATH.
Ejemplo:

```
C:\Program Files\PostgreSQL\16\bin
```

6.2 Crear usuario y base de datos

Desde PowerShell, ejecutar los siguientes comandos. Se solicitará la contraseña del usuario postgres definida durante la instalación:

```
psql -U postgres -c "CREATE ROLE turco WITH LOGIN PASSWORD 'clave_segura' CREATEDB;"  
createdb -U postgres -O turco repuestos_el_turco
```

Si el rol turco ya existe, se puede omitir el primer comando o usar otro usuario/contraseña. Los mismos datos deben quedar configurados luego en el archivo .env del backend.

6.3 Cargar la base de datos poblada

Desde la carpeta raíz del proyecto, ejecutar:

```
psql -U turco -d repuestos_el_turco -h localhost -f database\repuestos_el_turco_backup.sql
```

Este archivo crea las tablas y carga los datos de ejemplo de una sola vez, incluyendo productos, familias, subfamilias, usuarios, marcas de vehículos, modelos, motores y compatibilidades.

6.4 Configurar y levantar backend

```
cd backend  
Copy-Item .env.example .env  
notepad .env
```

En el archivo .env ajustar DATABASE_URL según el usuario, contraseña, host, puerto y nombre de base de datos.
Ejemplo con puerto 5432:

```
DATABASE_URL=postgresql://turco:clave_segura@localhost:5432/repuestos_el_turco
```

Si PostgreSQL está configurado en el puerto 5433, usar:

```
DATABASE_URL=postgresql://turco:clave_segura@localhost:5433/repuestos_el_turco
```

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Si PowerShell bloquea la activación del entorno virtual con un mensaje de ejecución de scripts deshabilitada, ejecutar una vez:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Si todo salió bien, el backend queda disponible en <http://localhost:8000> y la documentación interactiva de la API en <http://localhost:8000/docs>.

6.5 Levantar frontend

Abrir otra terminal, dejando el backend corriendo, y ejecutar desde la carpeta raíz del proyecto:

```
cd frontend
npm install
npm run dev
```

El sitio queda disponible en <http://localhost:8080>.

7. Instalación en Linux

Ejemplo para Debian/Ubuntu. En otras distribuciones se deben instalar los mismos paquetes con el gestor correspondiente.

7.1 Instalar PostgreSQL

```
sudo apt update
sudo apt install postgresql postgresql-contrib
sudo systemctl start postgresql
sudo systemctl enable postgresql
psql --version
```

7.2 Crear usuario y base de datos

```
sudo -u postgres psql -c "CREATE ROLE turco WITH LOGIN PASSWORD 'clave_segura' CREATEDB;"
sudo -u postgres createdb -O turco repuestos_el_turco
```

7.3 Cargar base de datos

```
psql -U turco -d repuestos_el_turco -h localhost -f database/repuestos_el_turco_backup.sql
```

7.4 Configurar y levantar backend

```
cd backend
cp .env.example .env
# editar .env y ajustar DATABASE_URL:
# DATABASE_URL=postgresql://turco:clave_segura@localhost:5432/repuestos_el_turco
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

7.5 Levantar frontend

```
cd frontend
npm install
npm run dev
```

8. Instalación en macOS

Se recomienda usar Homebrew para instalar PostgreSQL.

8.1 Instalar PostgreSQL

```
brew install postgresql@16
brew services start postgresql@16
psql --version
```

Si psql no queda en el PATH, agregar PostgreSQL al PATH. En Apple Silicon suele ser:

```
echo 'export PATH="/opt/homebrew/opt/postgresql@16/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

```
psql --version
```

8.2 Crear y cargar base de datos

```
createdb repuestos_el_turco  
psql -d repuestos_el_turco -f database/repuestos_el_turco_backup.sql
```

Si se prefiere usar usuario y contraseña específicos:

```
psql postgres -c "CREATE ROLE turco WITH LOGIN PASSWORD 'clave_segura' CREATEDB;"  
createdb -O turco repuestos_el_turco  
psql -U turco -d repuestos_el_turco -f database/repuestos_el_turco_backup.sql
```

8.3 Backend y frontend

```
cd backend  
cp .env.example .env  
# ajustar DATABASE_URL según la configuración usada  
python3 -m venv venv  
source venv/bin/activate  
pip install -r requirements.txt  
uvicorn app.main:app --reload
```

En otra terminal:

```
cd frontend  
npm install  
npm run dev
```

9. Variables de entorno del backend

El backend utiliza un archivo `.env` dentro de la carpeta `backend/`. Este archivo no debe subirse con credenciales reales al repositorio. El proyecto incluye `.env.example` como guía.

```
# Base de datos PostgreSQL
DATABASE_URL=postgresql://usuario:password@localhost:5433/repuestos_el_turco

# JWT
SECRET_KEY=tu_secret_key_super_segura
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=480

# Admin por defecto
ADMIN_USER=admin
ADMIN_PASS=password_admin

# CORS
CORS_ORIGINS=http://localhost:5173,http://localhost:3000

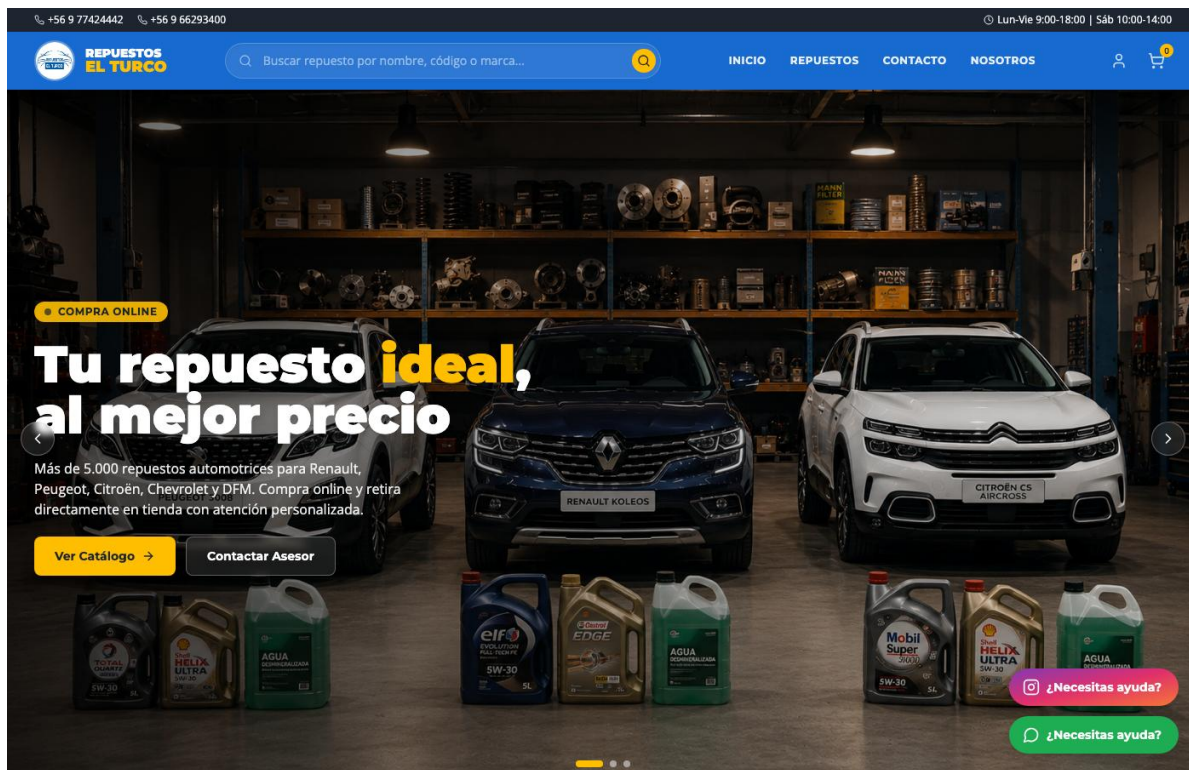
# Mercado Pago
MP_ACCESS_TOKEN=TEST-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
MP_PUBLIC_KEY=TEST-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
MP_WEBHOOK_URL=
FRONTEND_URL=http://localhost:8080
```

Nota: si se copia exactamente el archivo `.env.example`, el usuario administrador configurado será `admin / password_admin`. Si no existe `.env`, el código del backend tiene valores por defecto `admin / turco2026`. Para evitar confusiones, se recomienda definir explícitamente `ADMIN_USER` y `ADMIN_PASS` en `.env` y usar esos datos en el panel de administración.

10. Acceso al sistema

10.1 Sitio público

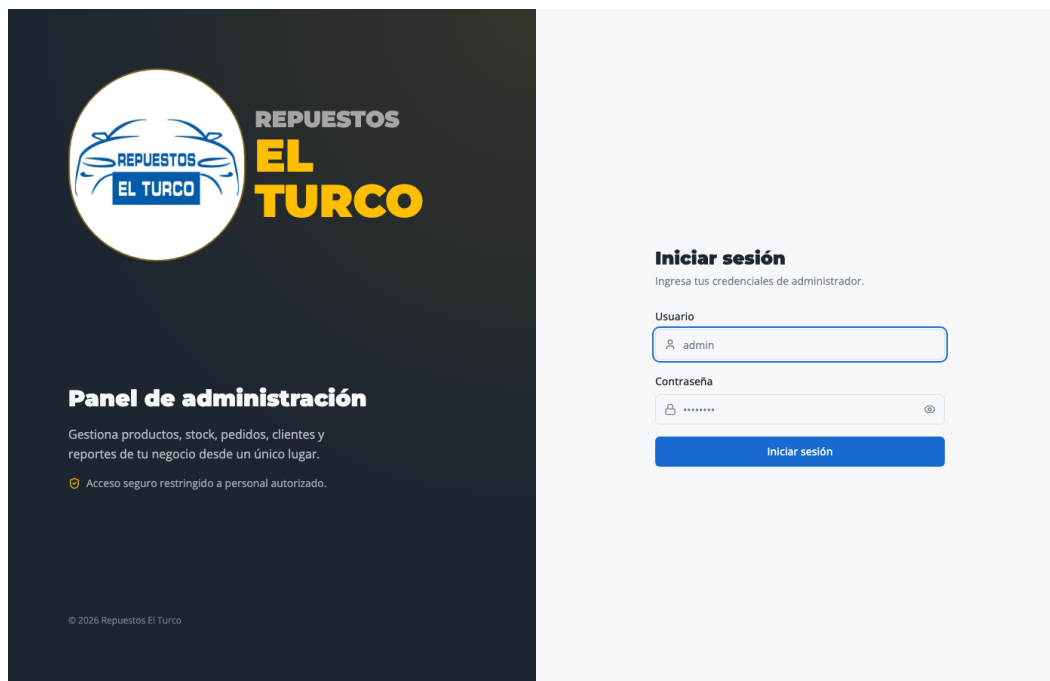
Acceder a `http://localhost:8080`. El sitio público no requiere autenticación y permite revisar catálogo, filtrar repuestos, agregar productos al carrito y finalizar compra.



Sitio público funcionando después de levantar backend y frontend.

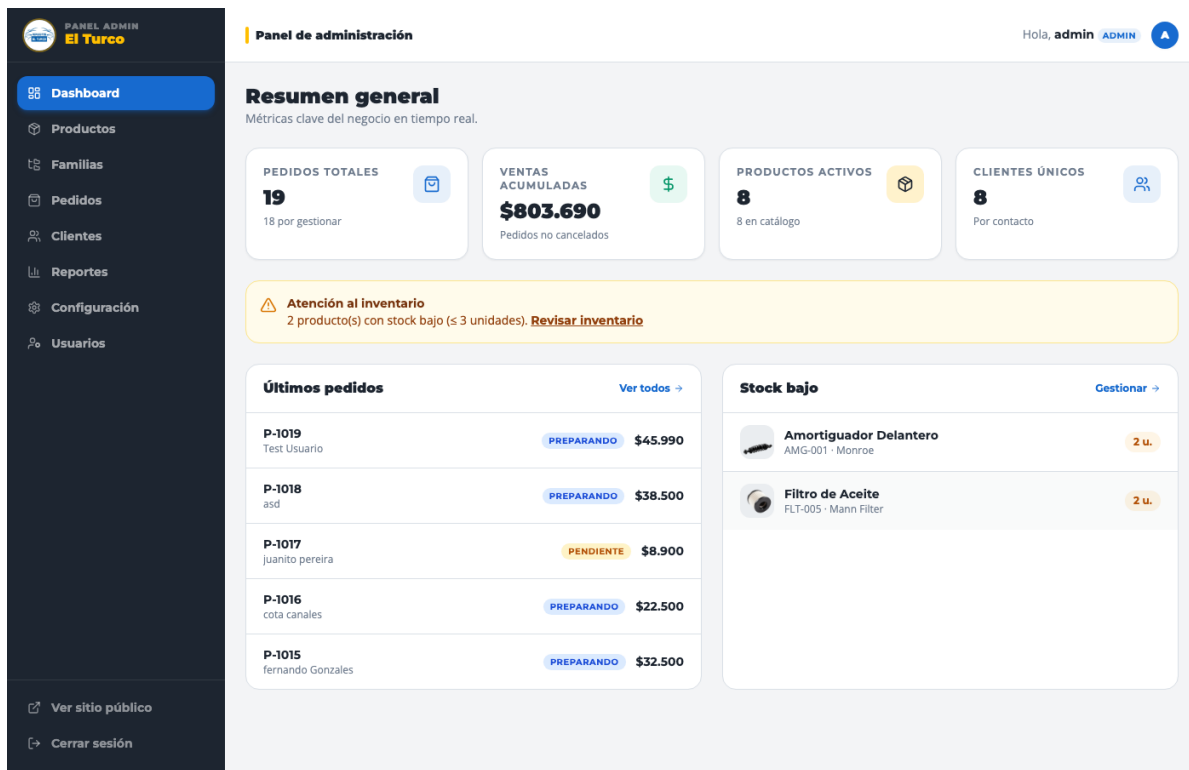
10.2 Panel de administración

Acceder a <http://localhost:8080/admin/login>. Las credenciales dependen del .env configurado en backend/.



Pantalla de inicio de sesión del panel de administración.

Después de iniciar sesión, el panel muestra resumen general, pedidos, ventas acumuladas, productos activos, clientes únicos y stock bajo.



Dashboard del panel de administración tras iniciar sesión correctamente.

11. Funcionalidades principales a verificar

- Catálogo público de productos.
- Filtro de repuestos por vehículo: marca, modelo, motor y año.
- Carrito de compras y finalización de pedido.
- Pago en línea mediante Mercado Pago Checkout Bricks.
- Confirmación de pedido.
- Panel de administración con dashboard.
- Gestión de productos, stock, familias, subfamilias, marcas y compatibilidades.
- Gestión de pedidos y cambio de estado.
- Gestión de usuarios administradores.
- Reportes de ventas.

12. Notas sobre Mercado Pago

- Las credenciales `MP_ACCESS_TOKEN` y `MP_PUBLIC_KEY` se obtienen desde Mercado Pago Developers.
- Para pruebas se deben usar credenciales que comiencen con `TEST-`.
- Con Checkout Bricks no se deben usar tarjetas reales en ambiente de prueba; se deben usar tarjetas oficiales de prueba.
- Para simular pago aprobado se puede usar titular `APRO` con tarjeta de prueba.
- `MP_WEBHOOK_URL` puede quedar vacío en local si no se usará un túnel público como ngrok.
- `FRONTEND_URL` debe apuntar al sitio donde corre el frontend, normalmente `http://localhost:8080`.

Tarjeta de prueba Mastercard: 5416 7526 0258 2580
Vencimiento: 11/30

CVV: 123
Titular: APRO
Email: un correo válido distinto al de la cuenta Mercado Pago

13. Problemas comunes

13.1 psql no se reconoce como comando

Agregar la carpeta bin de PostgreSQL al PATH y abrir una terminal nueva. Ejemplo de ruta en Windows:

```
C:\Program Files\PostgreSQL\16\bin
```

13.2 Error de conexión a PostgreSQL

- Verificar que PostgreSQL esté iniciado.
- Confirmar host, puerto, usuario y contraseña en DATABASE_URL.
- Revisar si el puerto real es 5432 o 5433.
- Probar conexión con psql antes de levantar el backend.

13.3 La base ya existe o quedó desactualizada

Si se desea partir desde cero, eliminar la base y volver a crearla antes de restaurar el backup. Ejecutar estos comandos desde una base distinta, por ejemplo postgres:

```
DROP DATABASE IF EXISTS repuestos_el_turco WITH (FORCE);  
CREATE DATABASE repuestos_el_turco;
```

13.4 “column ... does not exist” al iniciar backend

La base de datos existente no coincide con el modelo actual. Lo recomendado para instalación limpia es recrear la base y restaurar database/repuestos_el_turco_backup.sql. En actualizaciones puntuales se puede ejecutar el ALTER TABLE correspondiente.

13.5 El checkout de Mercado Pago no carga

- Verificar que MP_PUBLIC_KEY esté configurado en .env.
- Reiniciar el backend después de editar .env.
- Probar con navegador sin bloqueadores si el Brick no aparece.
- Los mensajes ERR_BLOCKED_BY_CLIENT hacia api.mercadolibre.com/tracks suelen ser telemetría bloqueada por el navegador y no necesariamente impiden el pago.

14. Checklist final de instalación

- Repositorio clonado correctamente.
- PostgreSQL instalado y funcionando.
- Base repuestos_el_turco creada.
- Backup database/repuestos_el_turco_backup.sql restaurado.
- Archivo backend/.env configurado.
- Dependencias del backend instaladas.
- Backend corriendo en http://localhost:8000.

- Frontend instalado y corriendo en `http://localhost:8080`.
- Sitio público abre correctamente.
- Panel admin permite iniciar sesión con el usuario configurado.
- Catálogo, carrito, pedidos y administración funcionan correctamente.